

Multi-Agent Anomaly Detection System

SYSTEM ARCHITECTURE DESIGN & COMMUNICATION PROTOCOL DOCUMENT

Deploy State: Active Live (Render Container)

AMQP Connection: CloudAMQP TLS (Port 5671)

Frameworks: FastAPI, Scikit-Learn, Pika, supervisord Author: Abhishek Tiwari, Developer

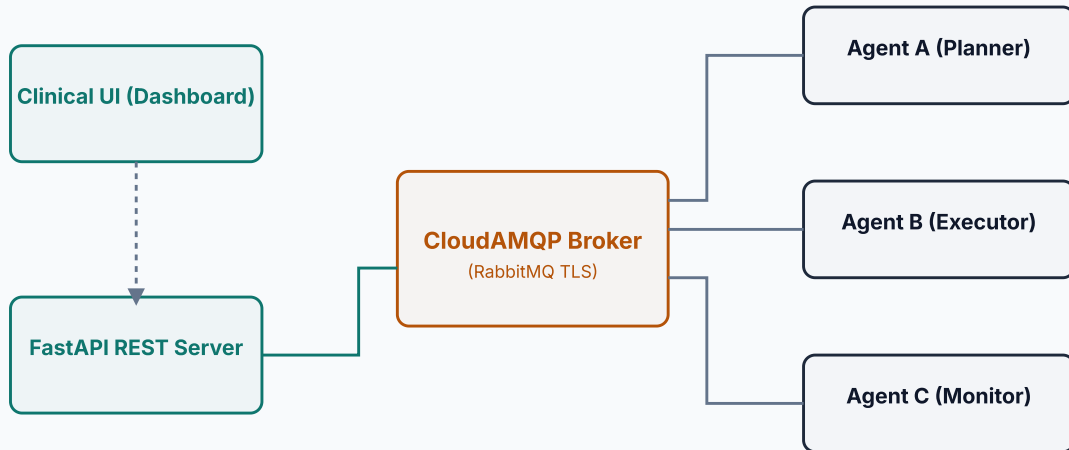
1. Executive Summary

The **Multi-Agent Anomaly Detection System** is a clinical-focused diagnostics monitor. It utilizes cooperative, decoupled agents to simulate patient vitals telemetry, train multivariate Isolation Forest models, and coordinate clinical warning alerts. Written in Python and served via FastAPI, the platform leverages asynchronous queues to ensure resilience under high compute loads. All telemetry statistics (Heart Rate, \$SpO_2\$, Blood Pressure, Temperature, Respiratory Rate, Glucose) are generated synthetically, providing a GDPR-compliant medical testing environment.

2. System Components & Architecture

Decoupling is managed through a central supervisord process coordinator running inside a single Render Web Service container. Nodes communicate externally over a managed TLS CloudAMQP broker:

- **FastAPI Server:** Serves a lightweight, light-themed clinical dashboard. Exposes endpoints to trigger analyses (`POST /tasks/analyze`), fetch status (`GET /tasks/{id}/status`), and display previous runs (`GET /tasks`).
- **Planner (Agent A):** Validates REST calls, creates execution parameters, publishes task messages, and tracks agent feedback metrics.
- **Executor (Agent B):** Performs compute-intensive operations. Simulates multi-dimensional vital signs, trains a multivariate Isolation Forest algorithm, maps anomalous records, and structures detailed JSON telemetry files.
- **Monitor (Agent C):** Supervises system state. It records progress percentages, tracks live node heartbeats, and routes critical alarms to clinicians.



Multivariate Machine Learning Advantage

Standard monitors flag warnings when a single vital (e.g., heart rate > 100) crosses a limit. The Isolation Forest algorithm trains on a multi-dimensional metric space, drawing isolation boundaries. This catches clinical emergencies (like early sepsis, where systolic BP falls while heart rate elevates) before any single vital crosses clinical thresholds.

3. Message Contract & JSON Schema

The system relies on a rigid envelope structure to guarantee reliable deserialization. All RabbitMQ payloads are encapsulated within a `MessageEnvelope` containing transaction tracing (Correlation ID), priority levels, and routing metadata:

```
{
  "message_id": "8e32ea70-e67c-473d-8182-d249f7e8a931",
  "sender_id": "agent-a",
  "receiver_id": "agent-b",
  "message_type": "TASK_ASSIGNMENT",
  "timestamp": "2026-06-26T07:18:40.125Z",
  "correlation_id": "session-42b781",
  "priority": 2,
  "routing_key": "task.agent_b",
  "payload": {
    "task_id": "task-df38190",
    "parameters": {
      "total_records": 7000,
      "contamination": 0.05,
      "random_seed": 42
    }
  },
  "metadata": {
    "retry_count": 0,
    "max_retries": 3,
    "routing_path": ["agent-a"]
  }
}
```

4. Communication Flow & Routing Keys

The messaging lifecycle utilizes the direct exchange `anomaly_exchange`. By binding queues to specific routing keys, we coordinate steps asynchronously:

Step	Source	Recipient	Routing Key / Channel	Message Type	Visual Result
1	Web UI	FastAPI	HTTP POST <code>/tasks/analyze</code>	-	User enters label, launches scan.
2	Agent A	Broker	<code>task.agent_b</code>	TASK_ASSIGNMENT	Planner allocates configuration.
3	Agent B	Agent A	<code>feedback</code>	TASK_ACCEPTED	Executor logs consumption.
4	Agent B	Agent C	<code>report</code>	TASK_PROGRESS (25% - 75%)	Sends simulation and training states.
5	Agent B	Agent C	<code>report</code>	TASK_PROGRESS (100%)	Compiles anomaly report variables.
6	Agent B	Agent C & A	<code>report & feedback</code>	TASK_COMPLETED	Pushes finished report details.
7	Agent C	Agent A	<code>feedback</code>	MONITOR_ALERT	Verifies severity alarms (normal/critical).

Heartbeat & Dead-Letter Queue (DLQ) Resilience

- **Heartbeats:** Agents publish tick messages to the broker every 30s. Agent C monitors timestamps and escalates offline alerts if any node is silent for > 90s.
- **DLQ Redirection:** If Agent B fails to process a task due to code errors, the Pika envelope increments `retry_count`. After 3 retries, the message is routed to the dead-letter exchange (DLX) for diagnostic capture.

5. AI Assistance Disclosure Statement

A clear boundary was maintained between ideation and coding phases:

- **Ideation Phase (Claude):** Claude was consulted for initial tech choices. It provided architecture comparisons that led to selecting RabbitMQ (TLS direct routing) and supervisord container management, framing the master plan.
- **Development Phase (Gemini / Antigravity):** Gemini performed all coding and debugging. This included overriding Pika's SSL verification settings for CloudAMQP compatibility, separating heartbeat channels to prevent socket corruption, building the light-themed CSS clinical dashboard, and solving test harness flakiness using isolated temporary queues.